Layer Normalisation

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. Normalisation in deep learning . . . 3
4. Batch normalisation, recapped as a table . . . 4
5. Why batch normalisation fails on sentences . . . 5
    5.1 Two padded sentences . . . 5
    5.2 A real batch of reviews . . . 6
6. Layer normalisation . . . 8
    6.1 How it works . . . 8
    6.2 What changes compared with batch normalisation . . . 11
7. Layer normalisation in the transformer . . . 11
    7.1 Normalising each word . . . 11
    7.2 Where it sits . . . 12
8. Summary . . . 14
9. Sources . . . 14
10. Key terms . . . 15

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Leads to:** Transformer encoder.
- **Compare with:** Batch normalisation.

1. Overview

Key point: Layer normalisation standardises each observation on its own, across its features, instead of each feature across the batch. In a transformer every word vector is normalised by its own mean and standard deviation, so the **padding** (G-1436) added to short sentences cannot distort the statistics of real words, as it does with batch normalisation.

The transformer has one component left before its full architecture: **normalisation** (G-1348). It uses **layer normalisation** (G-1054; Ba, Kiros and Hinton 2016), not the **batch normalisation** (G-266) of how batch normalisation works. Both standardise values to mean 0 and standard deviation 1 (a spread of about 1 around the mean) and then rescale them with learned numbers γ (a multiplier) and β (a shift). They differ in which numbers they average over (Figure 1).

batch normalisation

	z_1	z_2	z_3
obs. 1	7	5	4
obs. 2	2	3	4
obs. 3	6	5	1
obs. 4	3	5	4
obs. 5	2	4	6

one mean and std per node (column), over the 5 observations of the batch: z_1: $\mu = 4.0$, $\sigma = 2.10$

layer normalisation

	z_1	z_2	z_3
obs. 1	7	5	4
obs. 2	2	3	4
obs. 3	6	5	1
obs. 4	3	5	4
obs. 5	2	4	6

one mean and std per observation (row), over its 3 values: obs. 1: $\mu = 5.33$, $\sigma = 1.25$

Figure 1: The same batch of 5 observations and 3 nodes. Batch normalisation averages down each column (node); layer normalisation averages along each row (observation)

The sections below:

- recap normalisation and batch normalisation;

- show why batch normalisation fails on sentences that are padded;
- build layer normalisation;
- place it in the transformer.

2. Prerequisites

- How batch normalisation works: normalising a node over the mini-batch (a small group of observations), γ and β, moving averages.
- The standardization formula: $(x - \mu)/\sigma$ gives mean 0 and standard deviation 1.
- All words at once, with matrices: one contextual vector per word, of the same size as the embedding (the list of numbers that stands for a word).
- Cutting every review to 50 words (**padding** (G-1436) reviews to a common length) and **word embeddings** (G-2127).

3. Normalisation in deep learning

> **Key point:** We normalise the inputs and the hidden-layer activations so that their values stay in a fixed range. Training becomes more stable and faster.

Normalisation (G-1348) transforms data so that it has chosen statistical properties, usually mean 0 and variance 1. **Standardisation** (G-1874; subtract the mean, divide by the standard deviation) is one form; **min-max scaling** (G-1217) into a fixed range is another.

In a neural network we can normalise two things (Figure 2):

1. **the inputs**, the **features** (G-772; input variables, one column each of the data table) given to the first layer (see the fix: scale the inputs);
2. **the activations** of hidden layers, which are the inputs of the next layer.

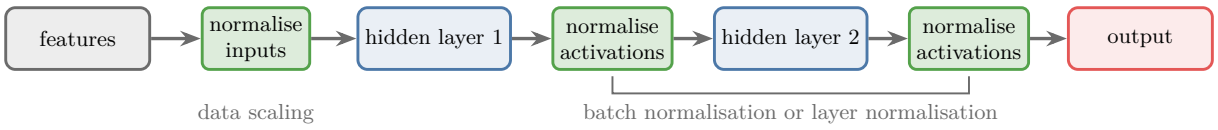


Figure 2: The two places a network normalises: the input features (data scaling) and the activations of each hidden layer before the next layer reads them (batch or layer normalisation)

The benefits, covered in why normalise inside the network, are more stable training, faster convergence, less **internal covariate shift** (G-963; each layer's inputs drifting as earlier weights change) and, for **batch normalisation** (G-266), a mild regularising effect. Batch normalisation and layer normalisation are two ways of normalising activations.

4. Batch normalisation, recapped as a table

> **Key point: Batch normalisation** (G-266) computes one mean and one standard deviation per node, over the observations of the batch: down each column of the table of z values.

Take a network with two input features and one hidden layer of 3 nodes. A batch of 5 **observations** (G-1374; one record each, one row of the data table) goes in. Each node computes its weighted sum z for every observation, giving the 5×3 table of Figure 1: column z_1 holds node 1's values 7, 2, 6, 3, 2.

Batch normalisation standardises each column separately, then scales by the node's γ and shifts by its β (see how batch normalisation works):

1. **In words:** for node 1, subtract the mean of its 5 values and divide by their standard deviation.
2. **Formula:** with μ_j and σ_j^2 the mean and variance (the average squared distance from the mean) of column j over the batch, and ϵ a tiny number (0.001) that avoids dividing by 0:

   $$\hat{z}_{n,j} = \frac{z_{n,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

   Then scale and shift:

   $$\gamma_j \hat{z}_{n,j} + \beta_j$$

3. **Example:** column z_1 holds 7, 2, 6, 3, 2.

   $$\mu_1 = (7 + 2 + 6 + 3 + 2)/5$$

   $$\mu_1 = 4$$

   The distances from the mean are 3, -2, 2, -1, -2. Their squares are 9, 4, 4, 1, 4:

   $$\sigma_1^2 = (9 + 4 + 4 + 1 + 4)/5$$

   $$\sigma_1^2 = 4.4$$

   $$\sigma_1 = \sqrt{4.4} = 2.10$$

   The first value becomes:

   $$\hat{z}_{1,1} = (7 - 4)/2.10$$

   $$\hat{z}_{1,1} = 1.43$$

   With $\gamma_1 = 1$ and $\beta_1 = 0$, their starting values, it stays 1.43 (Figure 3).

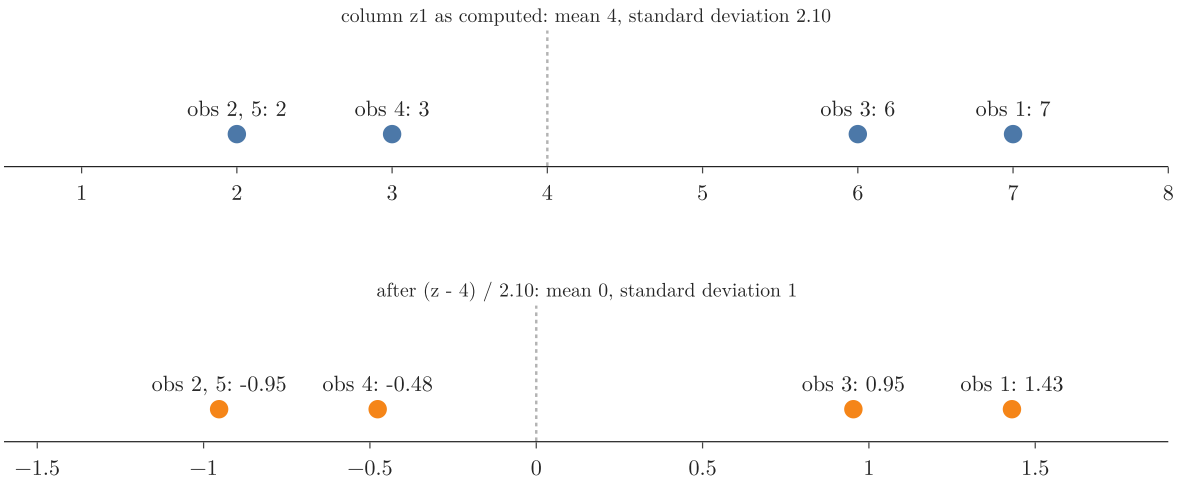


Figure 3: Node 1's five values before and after batch normalisation. Subtracting the mean 4 centres them on 0; dividing by 2.10 makes their standard deviation 1. The order and the relative gaps stay the same

The column means of z_2 and z_3 are 4.4 and 3.8. After batch normalisation every column has mean 0 and standard deviation 1. Keras' `BatchNormalization` returns the same table to within 3×10^{-7} (Notebook).

5. Why batch normalisation fails on sentences

> **Key point:** Sentences in a batch have different lengths, so the short ones are padded with zero vectors. Batch normalisation averages over all positions, **padding** (G-1436) included, so its mean and standard deviation describe mostly padding, not words.

5.1 Two padded sentences

> **Key point:** Stacking the word vectors of a padded batch and normalising each column mixes the zeros of the padding into every column's statistics.

In a transformer, the **normalisation** (G-1348) comes right after self-attention, which turns each word's embedding into a contextual vector of the same size. Self-attention can process several sentences at once as a batch, but every sentence in the batch must have the same number of words. Shorter sentences are filled up with **padding** (G-1436): extra positions holding zero vectors.

Take a batch of two sentences with 3-number vectors: "hi rahul" (2 words) and "how are you today" (4 words). The first sentence gets two padding rows. The batch is one tensor of shape $(2, 4, 3)$: 2 sentences, 4 positions, 3 numbers per position. **Batch normalisation** (G-266) treats each of the 3 numbers as a feature and stacks all 8 positions as rows (the Notebook's example vectors):

Sentence	Word	dim 1	dim 2	dim 3
1	hi	6.1	3.2	1.3
1	rahul	1.1	7.5	8.3
1	(padding)	0	0	0
1	(padding)	0	0	0
2	how	5.9	6.8	5.3
2	are	8.5	7.5	1.0
2	you	7.9	1.3	6.8
2	today	2.4	7.9	5.3

Each column's mean and standard deviation now include the two rows of zeros:

	dim 1	dim 2	dim 3
Mean over all 8 rows	3.99	4.28	3.50
Mean over the 6 real words	5.32	5.70	4.67

The padding pulls every mean down by about a quarter. Keras' `BatchNormalization`, applied to the $(2, 4, 3)$ tensor, uses exactly these 8 rows (Notebook). The padding zeros are not part of the data; they exist only so that the sentences fit in one tensor. Statistics computed with them do not represent the data.

5.2 A real batch of reviews

> **Key point:** In a batch of 32 real IMDB reviews padded to the longest one, 72% of all positions are **padding** (G-1436). As the padding grows, the batch-normalised value of a fixed real word keeps changing; its layer-normalised value does not move.

Real batches are far worse than the toy example. The first 32 reviews of the IMDB training set have between 43 and 888 words (median 175). Padded to the longest review, 72% of all positions are padding (Figure 4).

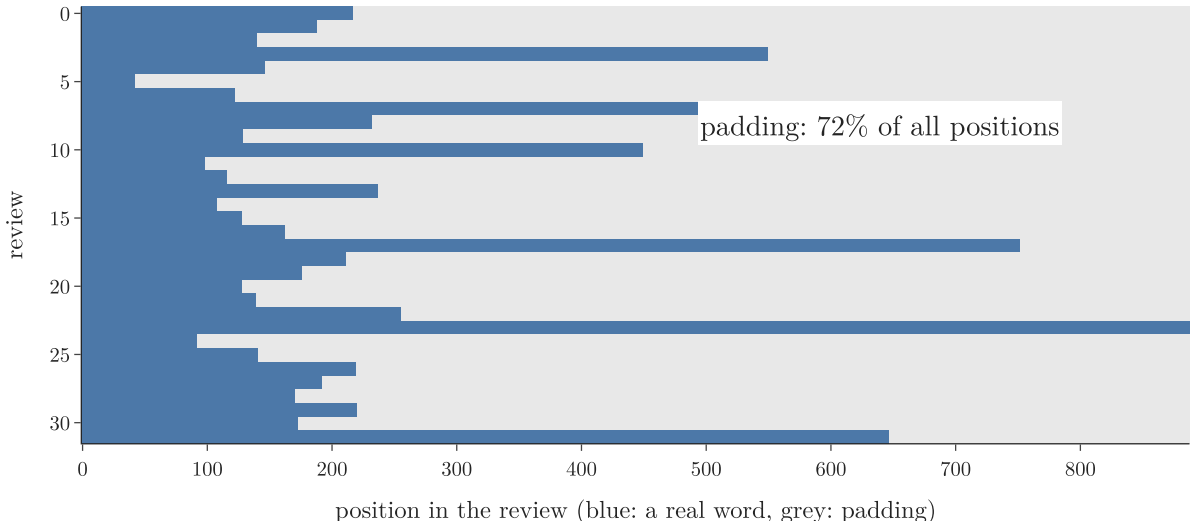


Figure 4: 32 real IMDB reviews padded to the longest one. Blue: positions holding a word; grey: padding

The Notebook gives every word an 8-number vector (a random embedding with mean 2 and standard deviation 1 stands in for the output of the layer before), sets the padding vectors to zero, and follows one real word: the first word of the first review, feature 1. For this feature, the mean over the real words is 2.05, but **batch normalisation** (G-266), counting the padding, uses 0.58.

Then the same 32 reviews are padded to more positions, as happens when a longer review joins the batch. Nothing about the real words changes:

Padded to	Share of padding	Batch norm mean	Batch-normalised value of the word	Layer-normalised value of the word
888	72%	0.58	1.76	1.63
1,000	75%	0.52	1.90	1.63
1,500	83%	0.34	2.43	1.63
2,000	87%	0.26	2.87	1.63
2,500	90%	0.21	3.24	1.63

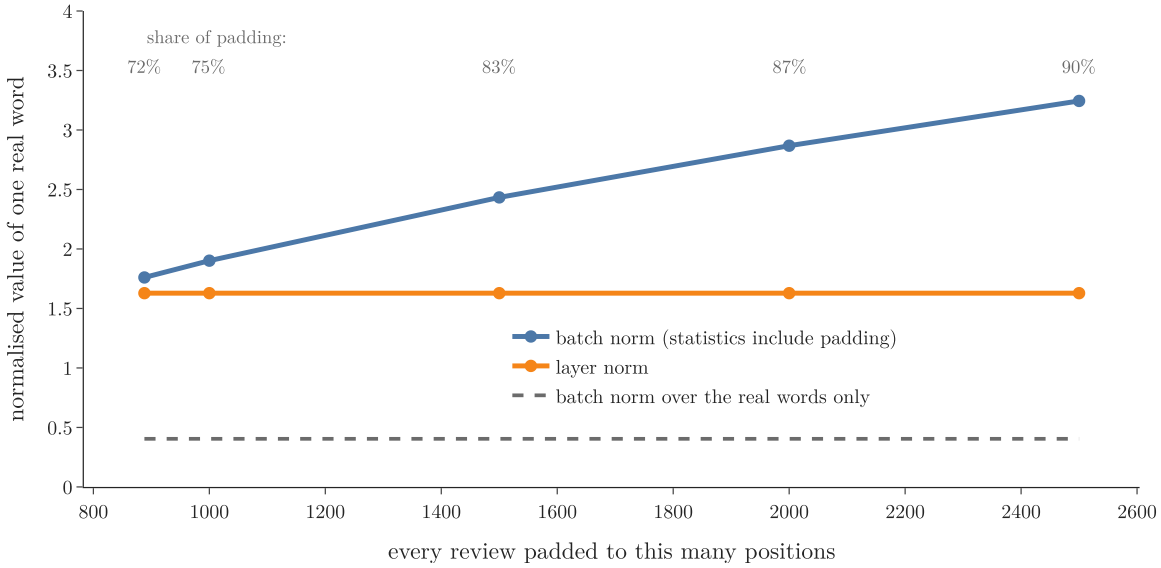


Figure 5: The same real word after normalisation, as the batch is padded to more positions. Batch normalisation (blue) drifts with the amount of padding, and is far from batch normalisation over the real words alone (dashed, 0.40). Layer normalisation (orange) does not change

The batch-normalised value of the same word nearly doubles, from 1.76 to 3.24, only because more zeros were added somewhere in the batch (Figure 5). Computed over the real words alone, batch normalisation would give 0.40. The layer-normalised value stays at 1.63, because layer normalisation looks only at the word's own vector (section 6). The two methods answer different questions, so 1.63 and 0.40 are not meant to agree; what matters is that one of them depends on the padding and the other does not.

Ba, Kiros and Hinton (2016, abstract and §3.1) name two more problems of batch normalisation that matter here: its effect "is dependent on the mini-batch size", and "it is not obvious how to apply it to recurrent neural networks", where sentences of different lengths would need separate statistics for every time step (every word position).

6. Layer normalisation

> **Key point:** Layer normalisation computes one mean and one standard deviation per observation, over its features: along each row of the table. Each feature still has its own γ and β.

6.1 How it works

> **Key point:** Same steps as **batch normalisation** (G-266), other direction: standardise each row, then scale and shift each feature with its γ_j and β_j.

Ba, Kiros and Hinton (2016, §3) "compute the layer normalization statistics over all the

hidden units in the same layer": all units of one layer share one mean and standard deviation, and different observations get different ones.

1. **In words:** for one observation, average its values over all the nodes of the layer, find their standard deviation, and standardise each value with these two numbers. Then multiply the value of node j by γ_j and add β_j.
2. **Formula:** observation n has values $z_{n,1}, \ldots, z_{n,H}$ over H nodes. The sign $\sum_{j=1}^{H}$ means "add the terms for $j = 1$, then $j = 2$, up to $j = H$". With $H = 3$ nodes:

$$\sum_{j=1}^{3} z_{n,j} = z_{n,1} + z_{n,2} + z_{n,3}$$

   The mean, the variance and the output of node j are:

$$\mu_n = \frac{1}{H}\sum_{j=1}^{H} z_{n,j}$$

$$\sigma_n^2 = \frac{1}{H}\sum_{j=1}^{H}(z_{n,j} - \mu_n)^2$$

$$\mathrm{LN}(z_{n,j}) = \gamma_j \frac{z_{n,j} - \mu_n}{\sqrt{\sigma_n^2 + \epsilon}} + \beta_j$$

3. **Example:** observation 1 of Figure 1 has $z = 7, 5, 4$.

$$\mu_1 = \frac{7 + 5 + 4}{3} = 5.33$$

$$\sigma_1^2 = \frac{1.67^2 + 0.33^2 + 1.33^2}{3} = 1.56$$

$$\sigma_1 = \sqrt{1.56} = 1.25$$

$$\hat{z}_1 = \frac{7 - 5.33}{1.25} = 1.34$$

$$\hat{z}_2 = \frac{5 - 5.33}{1.25} = -0.27$$

$$\hat{z}_3 = \frac{4 - 5.33}{1.25} = -1.07$$

   With $\gamma = 1$ and $\beta = 0$ these are the outputs. In general each value then gets its own feature's pair: 1.34 is scaled by γ_1 and shifted by β_1, -0.27 by γ_2 and β_2, -1.07 by γ_3 and β_3.

Every row now has mean 0 and standard deviation 1, while the columns no longer do. Keras' `LayerNormalization` gives the same table to within 3×10^{-7} (Notebook). Like `BatchNormalization`, it starts with $\gamma = 1$ and $\beta = 0$ and uses $\epsilon = 0.001$.

Python: The two layers on the same table. `BatchNormalization` needs `training=True` to use the batch's own statistics.

```
import numpy as np, keras

Z = np.array([[7, 5, 4], [2, 3, 4], [6, 5, 1],
              [3, 5, 4], [2, 4, 6]], dtype="float32")
bn = keras.layers.BatchNormalization()(Z, training=True)
ln = keras.layers.LayerNormalization()(Z)
# bn: each column has mean 0, std 1
# ln: each row has mean 0, std 1
```



Figure 6: The padded batch of section 5.1 normalised both ways. Batch normalisation sweeps down each column, and the padding zeros enter every mean; layer normalisation sweeps along each row, so each word is standardised with its own numbers and the padding rows stay 0 (with $\beta = 0$)

Figure 6 runs both on the padded batch of section 5.1. Watch the direction of the box: down the columns for batch normalisation, where the two **padding** (G-1436) rows join every average, and along the rows for layer normalisation, where they cannot.

6.2 What changes compared with batch normalisation

> **Key point:** Statistics per observation, so no dependence on the batch: the same computation in training and prediction, and no moving averages to store.

	Batch normalisation	Layer normalisation
Mean and variance computed over	the observations of the batch, per feature	the features of one observation
Number of means per batch of 5 observations, 3 nodes	3 (one per node)	5 (one per observation)
γ and β	one pair per feature	one pair per feature
Depends on the other observations of the batch	yes	no
Training and prediction	batch statistics, then moving averages	the same computation
Stored numbers per feature	4 (γ, β, moving mean, moving variance)	2 (γ, β)

Because each observation is normalised on its own, layer normalisation "performs exactly the same computation at training and test times" and works even with a batch of one (Ba, Kiros and Hinton 2016, abstract and §3). The Notebook confirms it: Keras' `LayerNormalization` gives identical outputs in training and prediction mode, while `BatchNormalization` gives outputs up to 1.10 apart on the same reviews: in prediction mode it uses its stored moving averages instead of this batch's mean and variance (Keras documentation, `BatchNormalization`).

7. Layer normalisation in the transformer

> **Key point:** In a transformer, one observation is one word vector. Each word's d_{model} numbers (d_{model} is the length of a word vector, 512 in the original transformer) are normalised by their own mean and standard deviation, so **padding** (G-1436) positions affect only themselves.

7.1 Normalising each word

> **Key point:** Real words are normalised with statistics from real numbers only; a zero padding row turns into β.

Return to the padded batch of section 5.1. Layer normalisation takes each of the 8 rows on its own and computes its mean and standard deviation from its 3 numbers. The row of "hi", $[6.1, 3.2, 1.3]$, has mean 3.53 and standard deviation 1.97, and becomes $[1.30, -0.17, -1.13]$ before γ and β, whatever the rest of the batch looks like. As Jurafsky and Martin put it, layer norm "is not applied to an entire transformer layer, but just to the embedding vector of a single token" (SLP3 §7.2.2).

A **padding** (G-1436) row is all zeros, so its mean is 0 and its variance is 0. It becomes zero:

$$\frac{0-0}{\sqrt{0+\epsilon}} = 0$$

After scaling and shifting it equals β:

$$\gamma \times 0 + \beta = \beta$$

The Notebook sets $\beta = [0.1, 0.2, 0.3]$ and gets exactly $[0.1, 0.2, 0.3]$ for both padding rows. The padding changes only its own rows. The real words never see it.

> **Extra:** The padding rows of section 5.1 are zero vectors by construction. Self-attention without a mask would not keep them at zero: a padding row has a zero query, so all its scores are 0, its attention weights are equal, and its output is the average of the value vectors. The Notebook gets weights of 0.25 for each of the 4 positions and an output equal to the mean of the value vectors. Keras' `MultiHeadAttention` therefore accepts an `attention_mask` that "prevents attention to certain positions", such as padding (Keras documentation, `MultiHeadAttention`).

7.2 Where it sits

> **Key point:** Each sub-layer of the transformer, self-attention or feed-forward, is followed by "add and norm": LayerNorm(x + Sublayer(x)).

Vaswani et al. (2017, §3.1) put a layer normalisation after each of the two sub-layers of an encoder block, the self-attention and the feed-forward network: "the output of each sub-layer is LayerNorm(x + Sublayer(x))". This step is called **add and norm** (G-172); Figure 7 shows both in one encoder block.

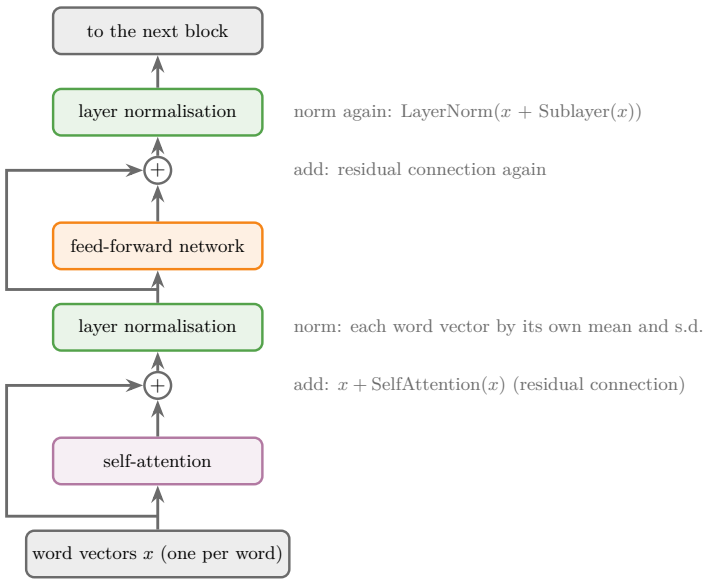


Figure 7: One transformer encoder block. After self-attention and after the feed-forward network, the sub-layer's input is added back (residual connection) and the sum is layer-normalised, word vector by word vector

The $x+$ part, a **residual connection** (G-1681; adding a layer's input back to its output), and the full block are explained in **add and norm** (G-172).

With $d_{\text{model}} = 512$, a `LayerNormalization` layer holds 1,024 parameters: γ and β for each of the 512 features, all trainable. A `BatchNormalization` layer of the same size holds 2,048, of which 1,024 are the non-trainable moving means and variances (Notebook).

> **Extra:** Two later variations are common. The most common transformer architecture today is the **prenorm** (G-1545) version, which applies the layer norm before the attention and feed-forward sub-layers; the original transformer is the **postnorm** version, with the norm after them (Jurafsky and Martin, SLP3 §7.2–7.3). Current models also often use **RMSNorm** (G-1696; Zhang and Sennrich 2019, §4, Eq. 4), which divides by the root mean square of the vector and skips subtracting the mean (SLP3 §7.2.2 and §7.8).

8. Summary

	Batch normalisation	Layer normalisation
Averages over	the batch, for each feature (down a column)	the features, for each observation (along a row)
In a transformer, one statistic per	embedding dimension, over all words and padding of the batch	word
Affected by padding	yes	no; a padding row only becomes β
Training vs prediction	different (moving averages)	identical
Parameters per feature	2 trainable + 2 non-trainable	2 trainable

- **Normalisation** (G-1348) keeps inputs and activations in a fixed range, for stable and fast training.
- **Padding** (G-1436) fills short sentences with zero vectors, because every sentence in a batch must have the same length; in a real IMDB batch, 72% of positions were padding, so padding is the normal case, not a rare one.
- **Batch normalisation** (G-266)'s statistics include the padding, so a real word's normalised value depends on how much padding the batch has (1.76 rising to 3.24 for the same word as padding grew).
- Layer normalisation standardises each word vector by its own mean and standard deviation, then applies a γ_j and β_j per feature, so padding cannot reach the real words (1.63 at every padding level) and training and prediction do the same computation.
- The transformer applies LayerNorm(x + Sublayer(x)) after each sub-layer, so every word vector leaves each sub-layer with its values back in a fixed range.
- That is why the transformer uses layer normalisation and not batch normalisation: its batches are padded sentences.

9. Sources

Built from

- CampusX, "Layer Normalization in Transformers | Layer Norm Vs Batch Norm", YouTube, https://www.youtube.com/watch?v=qti0QPdaelg

Other references

- Ba, J. L., Kiros, J. R. and Hinton, G. E. (2016). Layer Normalization. arXiv:1607.06450. Abstract; section 3 (statistics over all hidden units of a layer; no constraint on batch size); section 3.1 (sentences of different lengths in RNNs).
- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*. arXiv:1706.03762. Section 3.1 (LayerNorm(x + Sublayer(x))).
- Jurafsky, D. and Martin, J. H. *Speech and Language Processing*, 3rd ed. draft (19 August 2026). Chapter 7, sections 7.2–7.3 (layer norm on the vector of a single token; prenorm and postnorm; RMSNorm) and 7.8.

- Zhang, B. and Sennrich, R. (2019). Root Mean Square Layer Normalization. *NeurIPS 2019*. arXiv:1910.07467. Section 4, Eq. 4 (normalise by the root mean square alone, with no mean subtraction).
- Keras API documentation: `LayerNormalization`, `BatchNormalization` and `MultiHeadAttention` layers, keras.io/api/layers.
- Maas, A. L. et al. (2011). Learning Word Vectors for Sentiment Analysis. *ACL 2011*. The IMDB review dataset.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Normalisation (in a network) (G-1348)	Transforming values so they have chosen statistics, usually mean 0 and variance 1; in a network it is applied to the inputs and hidden activations so they stay in a fixed range and training is more stable and faster.
Layer normalisation (G-1054)	Putting each observation's numbers on one scale by itself, without depending on the batch: subtract the mean and divide by the standard deviation across its own features, then scale and shift each feature by learned γ_j and β_j.
RMSNorm (G-1696)	A simpler version of layer normalisation that divides each vector by its root mean square and skips subtracting the mean; many current transformers use it.
γ (gain) and β (shift) (G-10)	Two numbers per feature that layer normalisation learns in training: after the values are normalised, γ rescales them and β shifts them.
Padding (G-1436)	Adding zeros to sequences so that all have the same length.
Batch normalisation (G-266)	A layer placed between layers of a network that re-centres and re-scales its inputs during training; this keeps the activations away from the flat ends of sigmoid and tanh, which helps against vanishing gradients.
Word embedding (G-2127)	A learned real-valued vector for each word; words used in similar ways get nearby vectors.

Term	Meaning
Standardization (G-1874)	Scaling a column to mean 0 and standard deviation 1 (subtract the mean, divide by the standard deviation), so features measured on different scales become comparable.
Min-max scaling (G-1217)	Subtract the column's minimum and divide by its range, giving values from 0 to 1; the main normalization technique.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Internal covariate shift (G-963)	The way the values a layer receives keep changing their spread and centre during training, because the layers before it keep updating (the change in the distribution of a network's activations).
Observation (G-1374)	One record of a dataset: one row of the data table.
Add and norm (G-172)	Adds a sub-layer's input back to its output (a residual connection) and then layer-normalises the sum, so the signal and gradients pass through deep stacks and the numbers stay on one scale: LayerNorm(x + Sublayer(x)).
Residual connection (G-1681)	A path that skips a sub-layer and adds the sub-layer's input to its output; also called a skip connection.
Pre-LN transformer (prenorm) (G-1545)	A transformer variant with layer normalisation at the input of each sub-layer, inside the residual branch, instead of after the addition; it trains without warm-up.
ϵ (epsilon) (G-9)	A tiny number added to the variance to avoid dividing by 0; 0.001 in Keras.